

Generation

Define los Criterios para la Tarea de la IA y Escribe los Prompts

Paso 1: Definir los Criterios Específicos para la Tarea (Marco CLEAR)

Utiliza el marco CLEAR para definir los criterios específicos que tu herramienta de IA debe seguir al analizar los archivos de registro.

1. Relevancia del Contexto

- ¿Qué alcance debe tener la IA? ¿Hay tipos específicos de registros o palabras clave que se deben priorizar?
- Ejemplo: "Enfocarse en errores y advertencias relacionadas con la actividad del usuario; excluye los registros informativos a menos que estén marcados como críticos."

- Alcance: analizar cualquier registro de texto con formato «fecha hora NIVEL Componente - Mensaje»: servidor, actividad de usuario, depuración de aplicaciones y monitoreo del sistema.
- Prioridad 1: entradas ERROR (y CRITICAL/FATAL si existieran), por ejemplo DatabaseConnect, ServiceCrash, DiskFailure, DeadlockDetected, NullPointer.
- Prioridad 2: entradas WARNING/WARN (HighMemoryUsage, CPUOverload, DiskSpace, UnauthorizedAccess, PaymentFailure).
- Prioridad 3: entradas INFO y DEBUG solo cuando aportan contexto de un problema (por ejemplo, un ServerShutdown o un reintento posterior a un error), o si contienen palabras clave como «failed», «timeout», «unauthorized», «crashed», «retry», «deprecated».
- Se excluyen las entradas INFO/DEBUG rutinarias (inicios de sesión normales, backups exitosos, health checks) del listado de problemas; sí se pueden contar para dar contexto.

2. Coincidencia de Tono y Estilo del Lenguaje

- ¿Qué tono debe usar la IA? (por ejemplo, técnico, amigable)

Técnico, claro y directo, dirigido a ingenieros de soporte y administradores de sistemas, sin relleno ni lenguaje promocional.

Responde en el idioma del usuario (español por defecto) y conserva sin traducir los nombres de componentes, servicios, niveles y mensajes tal como aparecen en el registro.

Usa viñetas, tablas y encabezados fijos para que el resultado se pueda escanear rápido; las conclusiones van primero y el detalle después.

Distingue siempre los hechos (lo que dice el registro) de las hipótesis (posibles causas), marcando estas últimas como «Hipótesis».

3. Manejo de Errores y Escalamiento

- *¿Cómo debe responder la IA si hay incertidumbre o falta de datos?*

- Si el archivo no se puede leer, está vacío o no sigue un formato reconocible, lo dice y pide el archivo nuevamente o una muestra del formato; no inventa contenido.
- Si hay líneas que no puede interpretar (por ejemplo, las líneas de un stack trace en el Input 4 o niveles distintos como WARN frente a WARNING), las normaliza o las lista aparte como «líneas no clasificadas» con su número.
- Si faltan datos para concluir una causa (sin marcas de tiempo, sin ID de servicio, rango de tiempo muy corto), indica qué dato falta y presenta la conclusión como no confirmada.
- Escalamiento: recomienda escalar a un humano o al equipo responsable cuando detecta ERROR críticos (fallo de disco, caída de servicios de pago o autenticación), intentos de acceso no autorizado, o patrones que se repiten en 3 o más ocasiones sin resolución.
- Si el volumen supera lo que puede procesar en una sola respuesta, avisa, procesa por bloques y entrega un resumen consolidado.

4. Precisión y Fiabilidad de las Respuestas

- *¿Cómo asegurarás que las respuestas de la IA sean confiables y basadas en datos?*

- Todo conteo se calcula ejecutando código (Code Interpreter / Python) sobre el archivo, no «a ojo»; el GPT debe tener activada esa capacidad.
- Cada hallazgo cita evidencia: marca de tiempo, componente y mensaje (por ejemplo, «2024-04-25 09:10:35 ERROR DatabaseConnect»).
- Los totales se verifican entre sí: INFO + WARNING + ERROR (+ DEBUG) debe igualar el total de líneas analizadas; si no cuadra, se informa.
- No se usan conocimientos externos para afirmar causas; las causas probables se etiquetan como hipótesis.
- Validación humana: se comparan los resultados del GPT con un conteo manual de al menos un archivo (por ejemplo, el Input 1: 35 líneas, 15 INFO, 8 WARNING y 12 ERROR).

5. Eficiencia de las Respuestas

- *¿Qué tipos de preguntas debe ser capaz de manejar la IA? ¿Cómo debe adaptar sus respuestas según la complejidad de la pregunta?*

- **Preguntas simples** («¿cuántos errores hay?», «¿qué advertencias hay de memoria?»): respuesta corta de 1 a 3 líneas con el número y un ejemplo.
- **Preguntas intermedias** («resume el archivo», «lista los errores únicos con su frecuencia»): tabla estructurada con conteos.

- **Preguntas complejas** («¿hay patrones sistémicos?», «correlaciona errores entre servicios»): análisis por etapas con línea de tiempo, correlaciones, hipótesis y acciones recomendadas.
- Si el usuario sube varios archivos, primero resume cada uno y después ofrece una vista consolidada comparativa.
- Al final ofrece 1 o 2 siguientes pasos concretos (por ejemplo, «¿quieres que profundice en los errores de base de datos?»).

Paso 2: Escribir los Prompts (Marco TRACI)

Usar el marco TRACI para redactar 1-3 prompts que dirijan la herramienta de IA a realizar cada tarea.

- **Tarea 1:**

Prompt 1

Prompt 1 – Resumen general del archivo

Tarea: Analiza el archivo de registro adjunto y genera un resumen: total de líneas, rango de fechas, y cantidad de entradas por nivel (INFO, WARNING/WARN, ERROR, DEBUG).

Rol: Actúa como un ingeniero de confiabilidad de sitios (SRE) con experiencia en análisis de registros.

Audiencia: El resumen lo leerá un supervisor técnico que necesita entender el estado del sistema en menos de un minuto.

Creación: Entrega una tabla de conteos por nivel, seguida de un párrafo de máximo 4 líneas con los hallazgos principales.

Intención: Dar una visión rápida y verificable de la salud del sistema en el periodo registrado.

Prompt 2

Tarea: Lista todas las entradas ERROR y WARNING/WARN del archivo, agrupadas por nivel y ordenadas cronológicamente, con marca de tiempo, componente y mensaje. Omite las entradas INFO y DEBUG salvo que estén directamente relacionadas con un error.

Rol: Actúa como un analista de operaciones de TI que revisa incidentes.

Audiencia: El equipo de soporte que atenderá los incidentes y necesita ver rápidamente qué falló y cuándo.

Creación: Dos tablas (Errores y Advertencias) con las columnas: Hora, Componente, Mensaje, Servidor/Recurso afectado.

Intención: Ofrecer un inventario completo de los problemas para priorizar su atención.

Prompt 3

Tarea: Extrae los mensajes de error únicos del archivo. Si dos mensajes difieren solo en valores variables (hora, porcentaje, ID, IP, nombre de usuario), trátalos como el mismo mensaje y muestra la variante más representativa.

Rol: Actúa como un ingeniero de datos especializado en normalización de registros.

Audiencia: Un administrador que quiere un catálogo limpio de tipos de error, sin repeticiones.

Creación: Una lista numerada con: componente, mensaje normalizado y número de líneas en las que aparece.

Intención: Reducir el ruido del registro y mostrar cuántos tipos distintos de error existen realmente.

Repítelo para cada tarea.

Plantilla de Documentación

Instrucciones:

1. Introducción

- Propósito de la Guía

Explicar cómo se diseñó, configuró y usa el GPT personalizado «Log Analyzer» para que cualquier miembro del equipo pueda reproducirlo, mantenerlo y mejorarlo. Su finalidad es automatizar y agilizar el análisis de registros, que hoy se hace de forma manual.

- Alcance

La guía cubre desde la definición de objetivos hasta las pruebas y el uso diario. Aplica a registros de texto de servidor, actividad de usuario, depuración de aplicaciones y monitoreo del sistema. No cubre la integración con herramientas de monitoreo en tiempo real ni el análisis de registros binarios.

2. Descripción de la Herramienta

- Descripción

Log Analyzer es un GPT personalizado que recibe archivos de registro, los interpreta, y entrega resúmenes, conteos de problemas recurrentes y patrones de comportamiento, con evidencia verificable y recomendaciones de acción.
Características principales

- Características Principales

- Resumen de cada archivo: total de líneas, rango de fechas y conteo por nivel (INFO, WARNING, ERROR, DEBUG).
- Listado de errores, advertencias y mensajes de error únicos (normalizando valores variables).
- Conteo y ranking de problemas recurrentes, por tipo de error y por servidor/servicio.
- Detección de patrones y tendencias: ciclos de reinicio, degradación de recursos, ráfagas de errores y correlaciones entre eventos.
- Soporte para varios formatos y niveles (WARN y WARNING, líneas DEBUG, stack traces de varias líneas) y para varios archivos a la vez.
- Respuestas adaptadas a la complejidad de la pregunta, con conclusiones primero y escalamiento recomendado cuando corresponde.

3. Proceso de Desarrollo

Guía Paso a Paso

1. Establecimiento de Objetivos

Se tradujeron los objetivos de la misión en tres tareas medibles: (1) resumir registros con errores, advertencias y mensajes únicos; (2) contar problemas recurrentes; (3) identificar patrones y tendencias. Se añadió un cuarto requisito transversal: que la herramienta sea fácil de usar y soporte grandes volúmenes de registros diversos.

2. Recopilación de Datos

Se reunieron seis archivos de ejemplo de cuatro tipos: Server Log (Inputs 1 y 2), User Activity Log (Input 3), Application Debugging Log (Inputs 4 y 5) y System Monitoring Log (Input 6). Esta variedad permite probar la herramienta con distintos niveles, componentes y estilos de mensaje

3. Procesamiento previo de Datos

Se definieron reglas de normalización que el GPT aplica antes de contar: unificar WARN y WARNING; separar fecha, hora, nivel, componente y mensaje; unir las líneas de continuación (por ejemplo, los «at com.company...» del stack trace en el Input 4) con la entrada anterior; sustituir valores variables (porcentajes, IP, usuarios, IDs, milisegundos) por marcadores para agrupar mensajes equivalentes; y comprobar que los totales por nivel cuadren con el total de líneas.

4. Entrenamiento y Refinamiento de la Herramienta

No se reentrena un modelo: se configura el GPT con instrucciones (criterios CLEAR), los prompts TRACI como ejemplos, y se activa Code Interpreter para que los conteos se calculen con código. Después de cada prueba se ajustan las instrucciones: se añaden reglas cuando una salida es ambigua, se afinan los formatos de tabla y se refuerza la separación entre hechos e hipótesis.

Pruebas y evaluación. Se ejecutan los prompts de cada tarea con los seis archivos y se comparan los resultados con un conteo manual (por ejemplo, Input 1 debe dar 35 líneas: 15 INFO, 8 WARNING y 12 ERROR). Se evalúan exactitud de los conteos, ausencia de datos inventados, claridad del formato y tiempo de respuesta. Los fallos se registran y se corrigen en las instrucciones.

5. Pruebas y Evaluación

Se ejecutan los prompts de cada tarea con los seis archivos y se comparan los resultados con un conteo manual (por ejemplo, Input 1 debe dar 35 líneas: 15 INFO, 8 WARNING y 12 ERROR). Se evalúan exactitud de los conteos, ausencia de datos inventados, claridad del formato y tiempo de respuesta. Los fallos se registran y se corrigen en las instrucciones.

4. Principales Desafíos y Soluciones

- **Desafío 1**

formatos y niveles inconsistentes.

Los registros usan WARN en unos archivos y WARNING en otros, incluyen líneas DEBUG y tienen entradas de varias líneas (stack trace del Input 4). Solución: reglas de normalización en las instrucciones y un reporte de «líneas no clasificadas» para que nada se omita en silencio.

- **Desafío 2**

conteos imprecisos y posibles invenciones.

Los modelos de lenguaje pueden contar mal al leer listas largas o inferir causas sin base. Solución: obligar a calcular con código, citar la evidencia (marca de tiempo y componente), verificar que los totales cuadren y marcar las causas probables como «Hipótesis».

- **Consejos para el Éxito**

Empezar con un archivo pequeño y validar manualmente antes de escalar a muchos archivos.

- Procesar archivos muy grandes por bloques y consolidar al final.
- Mantener los prompts TRACI como biblioteca reutilizable y versionar las instrucciones del GPT.

- No cargar datos sensibles reales (IP, correos, nombres de usuario) sin anonimizarlos o sin autorización.
-

5. Casos de Uso Potenciales

- **Más Allá del Análisis de Registros**

Auditoría de seguridad (intentos de acceso no autorizado, fallos de inicio de sesión repetidos), análisis del comportamiento de usuarios (embudos de compra, errores de pago como los del Input 3), soporte a QA y depuración, y generación automática de reportes de incidentes.

- **Oportunidades Futuras**

Conectar la herramienta mediante API o acciones a sistemas de monitoreo para análisis casi en tiempo real; generar alertas automáticas; comparar periodos (semana contra semana); añadir gráficos de tendencias y detección de anomalías; y adaptar el GPT a otros formatos como JSON o syslog.

6. Instrucciones para el Usuario

- **Cómo Usar la Herramienta**

- Abre el GPT «Log Analyzer».
- Adjunta uno o varios archivos de registro (.txt o .log).
- Indica lo que necesitas: un resumen, un conteo, los patrones o un informe ejecutivo; puedes usar los prompts TRACI de este documento.
- Revisa el resultado y pide más detalle sobre los puntos que te interesen.

- **Entradas y Salidas**

Entradas: archivos de texto con una entrada por línea (fecha, hora, nivel, componente, mensaje) y, opcionalmente, una pregunta o un rango de tiempo. Salidas: tablas de conteos, listados de errores y advertencias, líneas de tiempo, patrones detectados, hipótesis y recomendaciones con su evidencia

- **Consejos para el Usuario**

Sé específico: «errores de base de datos entre 09:00 y 11:00» da mejores resultados que «analiza esto».

- Si algo parece incorrecto, pide al GPT que muestre las líneas del registro que respaldan su conclusión.
- Para varios archivos, pide primero el resumen individual y luego la comparación.
- Trata las hipótesis como puntos de partida para investigar, no como diagnósticos definitivos.

7. Escenarios de Ejemplo

Escenario 1

Server Log (Input 1), resumen y conteo de recurrencias

Pregunta del usuario: «Resume este archivo y dime cuáles son los problemas que más se repiten.»

Resultado esperado del GPT (verificable contra el archivo):

Problema	Veces	Detalle
DatabaseConnect (ERROR)	4	3 por «Timeout occurred» (09:10, 10:05, 11:05) y 1 por «Authentication failed» (09:35)
ServiceCrash (ERROR)	3	email-service (09:50 y 11:30) y payment-service (10:45)
FileNotFound (ERROR)	2	'app.config' no encontrado en /etc/app/ (09:20 y 11:15)
NetworkDisconnect (ERROR)	2	Conexión perdida con 'db-01' (10:35 y 11:45)
FilePermission (ERROR)	1	Permisos insuficientes sobre '/var/log/app.log' (10:20)

Totales: 35 entradas (15 INFO, 8 WARNING, 12 ERROR). Las 8 advertencias son 4 tipos que aparecen dos veces cada uno: DiskSpace, HighMemoryUsage, NetworkLatency y CPUOverload.

Patrones detectados: los errores de conexión a la base de datos se repiten aproximadamente cada hora; el servidor entra en tres ciclos de inicio y apagado (ServerStart/ServerShutdown) durante unas dos horas; el espacio en /dev/sda1 baja de 15 % a 12 %, la memoria de 'web-02' sube de 80 % a 85 % y la CPU de 'db-03' pasa de 90 % a 92 %. Hipótesis: los problemas de base de datos y de red podrían estar relacionados con las caídas de servicios. Recomendación: escalar la revisión de la base de datos y de 'db-01', y corregir la configuración 'app.config' que falta.

- **Escenario 2**

System Monitoring Log (Input 6), correlación de incidentes

Pregunta del usuario: «¿Hay problemas conectados en este registro? ¿Qué debería escalar?»

Resultado esperado del GPT: el archivo contiene 5 ERROR (DiskFailure, ServiceDown, BackupFailure, ProcessKill, ServiceFailure) y 6 WARNING (CPUUsage, MemoryLeak, HighTemp, LowDiskSpace, HighLatency, UnauthorizedAccess).

Cadena de eventos: el proceso 'data_aggregator' provoca un pico de CPU del 85 % a las 07:10 y es finalizado a las 08:25 por exceder su límite de memoria. A las 07:16 hay además una fuga de memoria en 'backup_service'. Nivel de confianza: medio. Hipótesis: un problema de gestión de memoria en procesos de datos.

Hardware y capacidad: fallo de lectura en '/dev/sdb' (07:15), temperatura de 75 °C en 'zone-3' (07:35) y solo 10 % de espacio libre en '/dev/sda1' (07:55). Recomendación: escalar el fallo de disco al equipo de infraestructura de inmediato.

Disponibilidad: 'user-auth-service' se cayó y se recuperó en unos 2 minutos; en cambio, 'report-generator' no logró iniciar (código 104) y el backup de 'server-002' falló por timeout de red; después se apaga 'server-002' por mantenimiento de emergencia.

Seguridad: intento de acceso no autorizado bloqueado desde la IP 198.51.100.23 (08:45), con reporte de incidente generado. Recomendación: escalar al equipo de seguridad para revisar si hay más intentos.